

US Patent & Trademark Office

Patent Public Search | Text View

United States Patent	12407718
Kind Code	B2
Date of Patent	September 02, 2025
Inventor(s)	Chen; Zhengzhang et al.

Incremental causal graph learning for attack forensics in computer systems

Abstract

A computer-implemented method for identifying attack origins is provided. The method includes detecting a trigger point from entity metrics data and key performance indicator (KPI) data, generating a learned causal graph by fusing a state-invariant causal graph with a state-dependent causal graph, backtracking from an attack detection point, via an incident backtrack and system recovery component, by using the learned causal graph to identify an attack origin when an intrusion or attack occurs, and displaying data relating to the attack origin on a visualization display for user analysis.

Inventors: Chen; Zhengzhang (Princeton Junction, NJ), Chen; Haifeng (West Windsor, NJ), Wang; Dongjie (Orlando, FL)

Applicant: NEC Laboratories America, Inc. (Princeton, NJ)

Family ID: 91583033

Assignee: NEC Corporation (Tokyo, JP)

Appl. No.: 18/359389

Filed: July 26, 2023

Prior Publication Data

Document Identifier	Publication Date
US 20240214414 A1	Jun. 27, 2024

Related U.S. Application Data

us-provisional-application US 63450989 20230309
us-provisional-application US 63442155 20230131
us-provisional-application US 63397955 20220815

Publication Classification

Int. Cl.: H04L9/40 (20220101); H04L41/0631 (20220101); H04L41/16 (20220101)

U.S. Cl.:

CPC H04L63/145 (20130101); H04L41/0631 (20130101); H04L41/16 (20130101);

Field of Classification Search

CPC: H04L (41/0631); H04L (41/16); H04L (63/145)

References Cited

U.S. PATENT DOCUMENTS

Patent No.	Issued Date	Patentee Name	U.S. Cl.	CPC
9736173	12/2016	Li et al.	N/A	N/A
10289841	12/2018	Tang et al.	N/A	N/A
10298607	12/2018	Tang et al.	N/A	N/A
10379849	12/2018	Jasinski	N/A	G06F 11/3636
2010/0161307	12/2009	Bharadwaj	703/21	G06F 11/3664
2018/0336349	12/2017	Zhang	N/A	G06F 21/554
2022/0070068	12/2021	Earhart	N/A	H04L 43/50
2022/0382614	12/2021	Chen et al.	N/A	N/A
2023/0069074	12/2022	Chen et al.	N/A	N/A
2023/0080424	12/2022	Yu	706/12	G06N 3/092
2023/0325269	12/2022	Gusat	714/2	G06F 11/0709
2024/0061740	12/2023	Chen	N/A	G06F 11/079

OTHER PUBLICATIONS

Wang et al., Disentangled Causal Graph Learning for Online Unsupervised Root Cause Analysis, arXiv preprint arXiv:2305.10638, pp. 1-11 (May 2023) (Year: 2023). cited by examiner

Wang et al., Incremental causal graph learning for online root cause analysis, In Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, pp. 2269-2278 (Aug. 2023) (Year: 2023). cited by examiner

Wang et al., Incremental Causal Graph Learning for Online Unsupervised Root Cause Analysis, arXiv preprint arXiv:2305.10638, pp. 1-10 (May 2023) (Year: 2023). cited by examiner

Dong, B., Chen, Z., Tang, L. A., Chen, H., Wang, H., Zhang, K., . . . & Li, Z. (Nov. 27, 2020). Anomalous event sequence detection. IEEE Intelligent Systems, 36(3), 5-13. cited by applicant

Lin, Y., Chen, Z., Cao, C., Tang, L. A., Zhang, K., Cheng, W., & Li, Z. (Oct. 17, 2018). Collaborative alert ranking for anomaly detection. In Proceedings of the 27th ACM International Conference on Information and Knowledge Management (pp. 1987-1995). cited by applicant

Dong, B., Chen, Z., Wang, H., Tang, L. A., Zhang, K., Lin, Y., . . . & Chen, H. (Nov. 6, 2017). Efficient discovery of abnormal event sequences in enterprise security systems. In Proceedings of the 2017 ACM on Conference on Information and Knowledge Management (pp. 707-715). cited by applicant

Wang, D., Chen, Z., Ni, J., Tong, L., Wang, Z., Fu, Y., & Chen, H. (Feb. 3, 2023). Hierarchical graph neural networks for causal discovery and root cause localization. arXiv preprint arXiv:2302.01987. cited by applicant

Cheng, W., Zhang, K., Chen, H., Jiang, G., Chen, Z., & Wang, W. (Aug. 13, 2016). Ranking causal anomalies via temporal and dynamical analysis on vanishing correlations. In Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (pp. 805-814). cited by applicant

Cai, L., Chen, Z., Luo, C., Gui, J., Ni, J., Li, D., & Chen, H. (Oct. 26, 2021). Structural temporal graph neural networks for anomaly detection in dynamic graphs. In Proceedings of the 30th ACM international conference on Information & Knowledge Management (pp. 3747-3756). cited by applicant

Luo, C., Chen, Z., Tang, L. A., Shrivastava, A., Li, Z., Chen, H., & Ye, J. (Jul. 19, 2018). TINET: learning invariant networks via knowledge transfer. In Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (pp. 1890-1899). cited by applicant

Primary Examiner: Zarka; David P

Background/Summary

RELATED APPLICATION INFORMATION (1) This application claims priority to Provisional Application No. 63/397,955 filed on Aug. 15, 2022, Provisional Application No. 63/442,155 filed on Jan. 31, 2023, and Provisional Application No. 63/450,989 filed on Mar. 9, 2023, the contents of all of which are incorporated herein by reference in their entirety.

BACKGROUND

Technical Field

(1) The present invention relates to root cause analysis, and, more particularly, to incremental causal graph learning for online root cause analysis.

Description of the Related Art

(2) Root Cause Analysis (RCA) aims to identify the underlying causes of system faults (e.g., anomalies, malfunctions, errors, failures) based on system monitoring data. RCA has been widely used in IT operations, telecommunications, industrial process control, etc., because a fault in these systems can greatly lower user experiences and cause losses.

SUMMARY

(3) A method for identifying attack origins is presented. The method includes detecting a trigger point from entity metrics data and key performance indicator (KPI) data, generating a learned causal graph by fusing a state-invariant causal graph with a state-dependent causal graph, backtracking from an attack detection point, via an incident backtrack and system recovery component, by using the learned causal graph to identify an attack origin when an intrusion or attack occurs, and displaying data relating to the attack origin on a visualization display for user analysis.

(4) A non-transitory computer-readable storage medium comprising a computer-readable program for identifying attack origins is presented. The computer-readable program when executed on a computer causes the computer to perform the steps of detecting a trigger point from entity metrics data and key performance indicator (KPI) data, generating a learned causal graph by fusing a state-invariant causal graph with a state-dependent causal graph, backtracking from an attack detection point, via an incident backtrack and system recovery component, by using the learned causal graph to identify an attack origin when an intrusion or attack occurs, and displaying data relating to the attack origin on a visualization display for user analysis.

(5) A system for identifying attack origins is presented. The system includes a processor and a memory that stores a computer program, which, when executed by the processor, causes the processor to detect a trigger point from entity metrics data and key performance indicator (KPI) data, generate a learned causal graph by fusing a state-invariant causal graph with a state-dependent causal graph, backtrack from an attack detection point, via an incident backtrack and system recovery component, by using the learned causal graph to identify an attack origin when an intrusion or attack occurs, and display data relating to the attack origin on a visualization display for user analysis.

(6) These and other features and advantages will become apparent from the following detailed description of illustrative embodiments thereof, which is to be read in connection with the accompanying drawings.

Description

BRIEF DESCRIPTION OF DRAWINGS

(1) The disclosure will provide details in the following description of preferred embodiments with reference to the following figures wherein:

(2) FIG. 1 is a block/flow diagram of an exemplary microservice intelligence system architecture, in accordance with embodiments of the present invention;

(3) FIG. 2 is a block/flow diagram of an exemplary root cause analysis module, in accordance with embodiments of the present invention;

(4) FIG. 3 is a block/flow diagram of an exemplary online trigger point detection system for streaming data, in accordance with embodiments of the present invention;

(5) FIG. 4 is a block/flow diagram of an exemplary disentangled graph learning system for incremental causal discovery, in accordance with embodiments of the present invention;

(6) FIG. 5 is a block/flow diagram of an exemplary workflow for root cause identification, in accordance with embodiments of the present invention;

(7) FIG. 6 is a block/flow diagram of an exemplary framework for detecting trigger points using monitoring entity metrics and key performance indicator (KPI) data, in accordance with embodiments of the present invention;

(8) FIGS. 7A-7C are block/flow diagrams of an exemplary framework for incremental disentangled causal graph learning, in accordance with embodiments of the present invention;

(9) FIG. 8 is a block/flow diagram of an exemplary online incident backtrack and system recovery engine, in accordance with embodiments of the present invention;

(10) FIG. 9 is a block/flow diagram of an exemplary incremental causal graph learning for attack forensics in an enterprise system, in accordance with embodiments of the present invention;

(11) FIG. 10 is a block/flow diagram of an exemplary architecture of automatic security intelligence, in accordance with embodiments of the present invention;

(12) FIG. 11 is a block/flow diagram of an exemplary processing system for online incident backtrack and system recovery, in accordance with embodiments of the present invention; and

(13) FIG. 12 is a block/flow diagram of an exemplary method for online incident backtrack and system recovery, in

accordance with embodiments of the present invention.

DETAILED DESCRIPTION OF PREFERRED EMBODIMENTS

(14) Previous root cause analysis (RCA) studies have focused primarily on developing effective offline methods for root cause localization. A key component of many data-driven offline RCA algorithms, especially for the causal discovery based RCA methods, is to learn the causal structure or causal graph that profiles the causal relations between system entities and system key performance indicators (KPIs) based on historical data, so that the operators can trace back the root causes based on the generated causal graph. For instance, conventional methods have utilized historical multivariate monitoring data to construct causal graphs using the conditional interdependence test, and then applied causal intervention to identify the root causes of a microservice system.

(15) However, the traditional offline causal discovery based RCA workflow has its limitations. First, for a new system's faults, the model needs to be retrained/rebuild from scratch. Second, the causal graph learning component is often time-consuming and requires a large amount of historical data to train a robust model. Third, it often requires the operators to manually initiate the RCA process when such operators observe a system fault. As a result, it's often too late to mitigate any damage or loss caused by a system fault. Based on that, the following questions can be asked:

(16) Is it possible to perform a causal discovery-based RCA task efficiently?

(17) How can a person identify the root causes as early as possible?

(18) Can the RCA algorithm be deployed online for the streaming data?

(19) If deployed online, can the time-consuming retraining of the RCA model from scratch be avoided every time a system fault occurs?

(20) In recent years, a very promising means for learning streaming data has emerged through the concept of incremental learning. Such incremental learning models rely on a compact representation of the already observed signals or an implicit data representation due to limited memory resources. In the exemplary RCA task, if the RCA model or causal graph for each batch of the streaming data can be incrementally updated, it virtually accelerates the RCA process. More importantly, the exemplary methods don't need to wait until the system fault occurs to trigger the RCA process. Thus, the exemplary methods can even be able to trigger an early RCA to mitigate the damages and losses. As a result, there exists a need for methods that can incrementally learn the RCA model and automatically trigger the RCA process.

(21) Enlightened by incremental learning, the exemplary methods aim to incrementally update the causal graph from streaming system monitoring data for accurately identifying root causes when a system failure or fault occurs.

Formally, given the initial causal graph learned from historical data, and the streaming system monitoring data including entity metrics and KPI data, the goal is to automatically initiate the RCA process when a system fault occurs, incrementally update the initial causal graph by considering each batch of data sequentially, and efficiently identify the top K nodes (e.g., system entities) in the updated causal graph that is most relevant to the system KPI. There are two challenges regarding this task:

(22) The first challenge is identifying the transition points between system states to initiate root cause analysis. As aforementioned, in traditional RCA, the operators often manually initiate the root cause procedure after a system fault occurs. To mitigate the damages or losses, in an online setting, it is needed to automatically detect the system state changes caused by the system fault and trigger the RCA process. The challenge is how to identify the transition points of system states early if the fault does not affect the system KPI, but only affects some root cause system entities at the early stage.

(23) The second challenge is to incrementally update the causal graph model in an efficient manner. After the transition/trigger points are detected, the old RCA model or causal graph cannot be directly applied to identify the root causes, since the old causal graph only includes the causal relations learned from the previous system state data. Although some inherent system dependencies do not change over time (e.g., system state-invariant causation), other causal dependencies may be highly dependent on the system state (e.g., system state-dependent causation). The challenge is how to identify the system state-invariant causation from the old model and quickly learn the state-dependent causation from the new batches of data for accelerating causal graph learning.

(24) To address these challenges, the exemplary methods introduce CORAL, a novel incremental causal graph learning framework, for online root cause localization. CORAL includes trigger point detection, incremental disentangled causal graph learning, and network propagation-based root cause localization. In particular, the first step of CORAL is to detect the transition points between system states in real-time based on system entity metrics and KPI data. To detect trigger points with less delay, the exemplary methods develop an online trigger point detection algorithm based on multivariate singular spectrum analysis and cumulative sum statistics. These points are then used to trigger incremental causal graph learning. Assuming that as the system state transitions, the underlying causal structure partially changes and evolves over time instead of shifting abruptly and significantly. Based on this assumption, the exemplary methods introduce an incremental disentangled causal graph learning model to efficiently learn causal relations by decoupling state-invariant and state-dependent causations. After that, the exemplary methods apply a random walk with restarts to model the network propagation of system faults to accurately identify root causes. The online root cause localization process terminates for the current system fault when the learned causal

graph and the generated root cause list converge.

(25) To summarize, the exemplary methods investigate the novel problem of online root cause localization. The exemplary methods propose to solve this problem by automatic trigger point detection and incremental causal structure learning. The exemplary methods introduce a principled framework referred to as CORAL, which integrates a new family of disentangled representation learning (e.g., causal graph disentanglement), online trigger point detection, and incremental causal discovery. System KPI is a monitoring time series that indicates the system status. For example, in a microservice system, latency is a KPI to measure the system status. The lower (higher) a system's latency is, the better (worse) its performance is.

(26) Entity metrics are multivariate time series collected by monitoring numerous system entities/components. For example, in a microservice system, a system entity can be a physical machine, container, virtual machine, pod, and so on. The system metrics include CPU utilization, memory consumption, disk IO utilization, etc. System entities with anomalous metrics can be the root causes of abnormal system latency/connection time, which is a sign of a system fault.

(27) Trigger point or system state change point is the time when the system transitions from one state to another. Real-world systems are dynamic. A system fault can cause a change in the system's status. As the state of a system varies, the underlying causal relationships between its components also change. Thus, to effectively identify root causes in an online setting, it is essential to learn different causal graphs in different states. From this perspective, the system state change points can be viewed as the triggers for updating the causal graph/online RCA model.

(28) Let $X = \{X_{\text{sub}.1}, \dots, X_{\text{sub}.N}\}$ denote N multivariate metric data. The i -th metric data is $X_{\text{sub}.i} = [x_{\text{sub}.1.\text{sup}.i}, \dots, x_{\text{sub}.T.\text{sup}.i}]$, where $x_{\text{sub}.t.\text{sup}.i} \in \mathbb{R}_{\text{sup}.M}$ is the observation of M system entities at time point t . To reduce notational clutter, the exemplary methods omit the metric index i and use X to represent $X_{\text{sub}.i}$ in the following sections. These observations are non-stationary, and the relationships between various system entities are dynamic and subject to change over time. It is assumed that the underlying state of the system can change when a system fault occurs, and the relationships between system entities in each state are represented by a directed acyclic graph (DAG). For simplicity, the exemplary methods illustrate using the system state transition from $s_{\text{sub}.p}$ to $s_{\text{sub}.p+1}$.

(29) The system KPI is y . The monitoring entity metric data of $s_{\text{sub}.p}$ is $\{\text{acute over } (X)\}_{\text{sub}.p} \in \mathbb{R}_{\text{sup}.p \times M}$, where p is the time length in the state $s_{\text{sub}.p}$. $G_{\text{sub}.p}$ represents the causal graph of $s_{\text{sub}.p}$, which includes nodes representing system KPI or entities, and edges representing causal relations. The data of state $s_{\text{sub}.p+1}$ comes one batch at a time, denoted by $\{\tilde{\text{acute over } (X)}\}_{\text{sub}.p+1} = [\tilde{X}_{\text{sub}.p+1.\text{sup}.1}, \dots, \tilde{X}_{\text{sub}.p+1.\text{sup}.L}]$, where the l -th batch $\tilde{X}_{\text{sub}.p+1.\text{sup}.l} \in \mathbb{R}_{\text{sup}.b \times M}$ and b is the length of each batch of data. The goal is to automatically trigger the RCA process when a system fault occurs, incrementally update $G_{\text{sub}.p}$ to $G_{\text{sub}.p+1}$ by considering each batch of data sequentially, and efficiently identify the top K nodes in the causal graph $G_{\text{sub}.p+1}$ that are most relevant to y .

(30) FIG. 1 is a block/flow diagram of an exemplary microservice intelligence system architecture **100**, in accordance with embodiments of the present invention.

(31) The agent **110** installs the JMeter in the microservice systems **140** to periodically send requests from the JMeter to the microservices systems **140** and collect system-level performance KPI data. The agent **110** also installs OpenShift/Prometheus to collect metrics data of all containers/nodes and applications/pods, e.g., central processing unit (CPU) usage and memory usage of a running pod during a period of time. The backend servers **120** receive the data from the agents **110**, pre-process them and send them to the analytic server **130**. The analytic server **130** runs the intelligent system management programs **150** to analyze the data. The root cause analysis engine **152** identifies the root causes of the system failure/faults. The intelligent system management programs **150** further includes risk analyzer **156**, log analyzer **158**, and failure/fault detector **154**. The techniques of the exemplary embodiments of the present invention are integrated in the root cause analysis engine.

(32) FIG. 2 is a block/flow diagram of an exemplary root cause analysis module, in accordance with embodiments of the present invention.

(33) The input **210** is provided to a first neural network **220**, a second neural network **222**, and a third neural network **224**, which perform deep neural network based causal analysis. The outputs of the first neural network **220**, the second neural network **222**, and the third neural network **224** are added with the learned attentions at block **230**.

Therefore, the low-level input and the learned attentions from the higher level are added. The output is provided to a fourth neural network **240**, a fifth neural network **242**, and a sixth neural network **244**. The fourth neural network **240**, the fifth neural network **242**, and the sixth neural network **244** perform deep neural network based causal analysis.

The output **250** is the root cause rankings and scores.

(34) FIG. 3 is a block/flow diagram of an exemplary online trigger point detection system **300** for streaming data, in accordance with embodiments of the present invention.

(35) The agent **110** (FIG. 1) collects the microservice data by employing the open-source JMeter and OpenShift/Prometheus. Two types of monitored data are used in the root cause analysis engine, that is, the JMeter report data of the whole system and the metrics data of the running containers/nodes and the applications/pods.

(36) The JMeter data includes the system performance KPI information such as elapsed time, latency, connect time, thread name, throughput, etc.

(37) It is in the following format: timeStamp, elapsed, label, responseCode, responseMessage, threadName, dataType, success, failureMessage, bytes, sentBytes, grpThreads, allThreads, URL, Latency, IdleTime, Connect_time.

(38) The exemplary methods use the Latency and Connect_time as two key performance KPIs of the whole microservice system. The Latency measures the latency from just before sending the request to just after the first chunk of the response has been received, while Connect_time measures the time it took to establish the connection, including a secure sockets layer (SSL) handshake. Both Latency and Connect_time are time series data, which can indicate the system status and directly reflects the quality of service, that is, whether the whole system had some failure events occur or not, because the system failure would result in the latency or connect time significantly increasing.

(39) The metrics data, on the other hand, includes a number of metrics which indicate the status of a microservice's underlying component/entity. The underlying component/entity can be a microservice's underlying physical machine/container/virtual machine/pod. The corresponding metrics can be the CPU utilization/saturation, memory utilization/saturation, or disk IO utilization. All these metrics data are essentially time series data. An anomalous metric of a microservice's underlying component can be the potential root cause of an anomalous JMeter Latency/Connect_time, which indicates a microservice failure.

(40) For the online trigger point detection, the exemplary methods first construct the initial system state space **310** by the time-lagged trajectory matrix H (**315**) based on the data X . Then, the exemplary methods learn a robust subspace $\text{.Math.}.\text{sup.}T$ of the system state by non-linear kernel-based subspace learning **320**. After that, the exemplary methods detect the system change by calculating the distance (**325**) between the new batch of data and the subspace, which can be represented as:

$$(41) D(t) = \text{.Math.} \hat{U}^T @ X(t-L+1:t) \text{.Math.} \frac{2}{F} - c$$

(42) where $@$ refers to a matrix multiplication, $X(t-L+1:t)$ is the new batch of data, and c is an adjustable parameter. Then, $D(t)$ is used to conduct a cumulative sum chart (CUSUM) (**330**) for detecting root causes based on the following equation:

$$(43) y(t) = \max\{y(t-1) + D(t), 0\}, y(0) = 0.$$

(44) If one trigger point is detected, the time-lagged trajectory matrix is updated (**340**), which means that the system has achieved to a new state.

(45) FIG. 4 is a block/flow diagram of an exemplary disentangled graph learning system **400** for incremental causal discovery, in accordance with embodiments of the present invention.

(46) For the incremental causal discovery, there are two learning objectives, that is, to keep system inherent causal relations and to capture system state dependent causal relations. A novel incremental causal discovery framework is presented.

(47) The main modules in this framework are an encoder **410**, a graph generation layer **420**, and a prediction layer **430**. The input of the encoder **410** includes previous system state data $S.\text{sub.}k$, the new batch of data in the next system state $S.\text{sub.}k+1.\text{sup.}t$, and the causal graph of the previous system state $G.\text{sub.}k.\text{sup.}t$. To obtain the system state invariant embedding, the encoder **410** converts $S.\text{sub.}k$, $S.\text{sub.}k+1.\text{sup.}t$, and $G.\text{sub.}k.\text{sup.}t$ into the embedding $z.\text{sub.}k.\text{sup.}t$.

(48) The calculation process is as follows:

$$(49) z_k^t = \text{Encoder}(S_k, S_{k+1}^t, G_t^k)$$

(50) To obtain the system state dependent embedding, the encoder **410** converts $S.\text{sub.}k+1$ and $G.\text{sub.}k.\text{sup.}t$ into the embedding $\{\tilde{z}\}.\text{sub.}k.\text{sup.}t$.

(51) The converting process can be represented as follows:

$$(52) \tilde{z}_k^t = \text{Encoder}(S_{k+1}^t, G_t^k)$$

(53) Then, the exemplary methods reconstruct the causal graph $G.\text{sub.}t.\text{sup.}k$ based on $z.\text{sub.}k.\text{sup.}t$ and $\{\tilde{z}\}.\text{sub.}k.\text{sup.}t$ respectively by decoder 1 (**415**) and decoder 2 (**417**).

(54) The process can be represented as follows:

$$(55) \hat{G}_k^t = \text{sigmoid}(z_k^t @ \text{transpose}(z_k^t)) \hat{G}_k^t = \text{sigmoid}(\tilde{z}_k^t @ \text{transpose}(\tilde{z}_k^t))$$

(56) Where $@$ refers to the matrix multiplication operation and transpose refers to the matrix transpose operation. After that, the exemplary methods propose a new graph generation layer **420** to generate the causal graph at the next batch based on $z.\text{sub.}k.\text{sup.}t$ and $\{\tilde{z}\}.\text{sub.}k.\text{sup.}t$, which can be formulated as follows:

$$(57) G_k^{t+1} = \text{relu}(\tanh(\hat{G}_k^t @ \hat{G}_k^t - \hat{G}_k^t @ \hat{G}_k^t))$$

(58) This layer **420** makes the learned causal graph become sparse and reduces the computational cost. The learned new causal graph is input into the prediction layer to fit the current batch of data of new system state for adjusting the learned causal relations.

$$(59) \text{.Math.} \hat{S}_{k+1}^t = \text{Prediction}(S_{k+1}^t, G_{k+1}^{t+1})$$

(60) The optimization objective of the whole process is as follows:

(61) $L_{\text{total}} = L_1 + L_2 + L_3$ $L_1 = \text{cross_entropy}(G_k^t, \hat{G}_k^t) + KL\{q(z_k^t) \cdot \text{Math.} \cdot \text{Math.} p(z_k^t | S_k, S_{k+1}^t)\}$
 $L_2 = \text{cross_entropy}(G_k^t, \hat{G}_k^t) + KL\{q(z_k^t) \cdot \text{Math.} \cdot \text{Math.} p(z_k^t | S_{k+1}^t)\}$ $L_3 = \text{MSE}(S_{k+1}^t, f(G_k^{t+1}, \hat{S}_{k+1}^t))$

(62) After the model converges, the causal graph is input into a root cause analysis algorithm to output the top-k pods/applicates as root causes of system failures.

(63) Therefore, the exemplary methods propose CORAL, a novel online RCA framework that can automatically trigger the RCA process and incrementally update the RCA model. CORAL includes trigger point detection, incremental disentangled causal graph learning, and network propagation-based root cause localization. The trigger point detection component aims to detect system state transitions automatically and in near real-time. To achieve this, the exemplary methods develop an online trigger point detection approach based on multivariate singular spectrum analysis and cumulative sum statistics. To efficiently update the RCA model, the exemplary methods propose an incremental disentangled causal graph learning approach to decouple the state-invariant and state-dependent information. After that, CORAL applies a random walk with restarts to the updated causal graph to accurately identify root causes. The online RCA process terminates when the causal graph and the generated root cause list converge.

(64) FIG. 5 is a block/flow diagram of an exemplary workflow for root cause identification, in accordance with embodiments of the present invention.

(65) The microservice management component includes a root cause identifier 502, an online root cause analyzer 504, and a data collection agent 506.

(66) The online root cause analyzer 504 enables online trigger point detection 510, incremental causal discovery 512, and root cause localization 514.

(67) The online trigger point detection 510 involves nonlinear subspace learning 520.

(68) The incremental causal discovery 512 involves disentangle graph learning 522.

(69) The root causal localization 514 involves incremental causal updates 524.

(70) FIG. 6 is a block/flow diagram of an exemplary framework 600 for detecting trigger points using monitoring entity metrics and key performance indicator (KPI) data, in accordance with embodiments of the present invention.

(71) FIG. 6 shows the proposed framework CORAL for online root cause localization. Specifically, CORAL addresses the discussed challenges with the following contributions:

(72) Online trigger point 610 detection detects transition points between system states and triggers incremental causal structure learning.

(73) Incremental disentangled causal graph learning is a novel disentangled graph learning method that enables efficient causal structure learning by decoupling state-invariant and state-dependent causations.

(74) Network propagation-based root cause localization models the network propagation of system faults to accurately identify the most possible root causes.

(75) The goal is to detect the trigger points 610 by integrating both entity metrics and system KPI data. The exemplary embodiments model the underlying dynamics of system entity metrics and KPI data (e.g., multivariate time series observations) through the Multivariate Singular Spectrum Analysis (MSSA) model. For simplicity, the exemplary methods add the system KPI, y, as one dimension to the metric data, X, to illustrate the model.

(76) Specifically, given monitoring metric data X, the exemplary methods first construct the base matrix, denoted by Z.sub.x, by using the previous T.sub.0 records. The requirement for the initial value of T.sub.0 is that no system state transition occurs in the time segment $t \leq T_{\text{sub.0}}$.

(77) The singular vectors of Z.sub.x are then grouped into two matrices .Math..sub.0 and .Math..sub.⊥.

(78) The exemplary methods estimate the pre-change subspace $\{\text{circumflex over (L)}\}_{\text{sub.0}}$ as follows:

(79) $\hat{L}_0 = \text{span}(\hat{U}_0)$ (1)

(80) Meanwhile, let $\{\text{circumflex over (L)}\}_{\text{sub.⊥}} = \text{span}(\text{Math..sub.⊥})$ be the orthogonal complement of the subspace $\{\text{circumflex over (L)}\}_{\text{sub.0}}$. After that, for the new data $t > T_{\text{sub.0}}$, the exemplary methods build the L-lagged matrix $X(t-L+1:t)$. The Euclidean distance between the L-lagged matrix and the estimated subspace $\{\text{circumflex over (L)}\}_{\text{sub.0}}$ is computed as the detection score, which can be defined as:

(81) $D(t) = \text{Math.} \frac{\|\hat{U}_\perp^T X(t-L+1:t) - c\|_F^2}{F} - c$ (2)

(82) where $c \geq 0$ is the shift-downwards constant. Moreover, the cumulative detection score is computed using the cumulative sum (CUSUM) statistics, which can be defined as:

(83) $y(t) = \max\{y(t-1) + D(t), 0\}$ $y(T_0) = 0$ (3)

(84) if the $y(t)=0$, the exemplary methods proceed to check the next time point. Otherwise, the change point is identified when $y(t) > h$, where h is a predefined threshold.

(85) This process can be defined as:

(86) $\hat{t} = \inf_{t > T_0} \{t \mid y(t) \geq h\}$

(87) The change point $\{\text{circumflex over (t)}\}$ is the trigger for incremental causal graph learning. To recheck the next change point, the base matrix is updated with the time segment $X(\{\text{circumflex over (t)}\}, \{\text{circumflex over (L)}\})$

(τ) + T.sub.0-1). This model can detect trigger points in nearly real-time.

(88) FIGS. 7A-7C is a block/flow diagram of an exemplary framework **700** for incremental disentangled causal graph learning, in accordance with embodiments of the present invention.

(89) After a trigger point is detected, the exemplary methods propose a disentangled causal graph learning model by integrating state invariant and state-dependent information, as well as incremental learning.

(90) The system state encoder **700** aims to decouple state-invariant and state-dependent information in order to produce corresponding embeddings. State-invariant and state-dependent decoders aim to reconstruct and refine state-invariant and state-dependent causal relations, respectively. The causal graph fusion module aims to fuse state-invariant and state-dependent causation in order to obtain the new causal graph.

(91) The goal of the system state encoder **700** is to aggregate the information from system state data and the corresponding causal graph. The exemplary methods use the k-th batch data **702** to illustrate the design. For simplicity, it is assumed that the previous state data $\{\tilde{X}\}_{\text{sub.p}}$ and the batch of new state data $\tilde{X}_{\text{sub.p}+1.\text{sup.k}}$ both have the system KPI y as one dimension.

(92) Given $\{\tilde{X}\}_{\text{sub.p}}$, $\tilde{X}_{\text{sub.p}+1.\text{sup.k}}$ and the causal graph at the k-1 batch $G_{\text{sub.p}+1.\text{sup.k}-1}$, the exemplary methods aim to integrate their information into state-invariant and state-dependent embeddings, respectively. First, a fully-connected linear layer is employed to preserve the information of $\{\tilde{X}\}_{\text{sub.p}}$ into a latent representation $U_{\text{sub.p}}$, which can be defined as:

$$(93) \quad U_p = \tilde{X}_p \cdot \text{Math. } W_p + b_p \quad (5)$$

(94) where $W_{\text{sub.p}}$ and $b_{\text{sub.p}}$ are the weight matrix and bias item of the linear layer, respectively.

(95) Then, to track the information change in the state p+1's data batch, the exemplary methods employ a recurrent function $f(\cdot, \cdot)$. The function $f(\cdot, \cdot)$ takes $\tilde{X}_{\text{sub.p}+1.\text{sup.k}}$ and the previous hidden state $H_{\text{sub.p}+1.\text{sup.k}-1}$ as inputs and outputs a latent representation $H_{\text{sub.p}+1.\text{sup.k}}$, which is defined as:

$$(96) \quad H_{p+1}^k = f(X_{p+1}^k, H_{p+1}^{k-1}) \quad (6)$$

(97) A long short-term memory network (LSTM) **704** is employed to implement $f(\cdot, \cdot)$. Due to the computational overhead, the exemplary methods do not use the passed batch of data to update the new causal graph. Thus, it is effective to track the causal dynamics using the LSTM module.

(98) In addition, because state-invariant causal relations would be affected by both the previous state data and the new batch of data, whereas state-dependent causal relations are only affected by the new batch of data, to obtain the state-invariant embedding $\{\tilde{Z}\}_{\text{sub.p}+1.\text{sup.k}}$, the exemplary methods first concatenate $U_{\text{sub.p}}$ and $H_{\text{sub.p}+1.\text{sup.k}}$ together (**710**), and then map it to the causal graph $G_{\text{sub.p}+1.\text{sup.k}-1}$ as the node embeddings. After that, the exemplary methods employ the mapping function $g(\cdot, \cdot)$ to convert the attributed graph into the state-invariant embedding, which is defined as:

$$(99) \quad \tilde{Z}_{p+1}^k = g(A_{p+1}^{k-1}, \text{Concat}(U_p, H_{p+1}^k)) \quad (7)$$

(100) where $A_{\text{sub.p}+1.\text{sup.k}-1}$ represents the adjacency matrix of $G_{\text{sub.p}+1.\text{sup.k}-1}$ and Concat represents the concatenated operation. To implement the function $g(\cdot)$, the exemplary methods employ a variational graph autoencoder (VGAE) **712**. VGAE **712** embeds all information into an embedding space that is smooth and continuous. This is helpful for capturing the causal dynamics between different system entities. To obtain the state-dependent embedding $\hat{Z}_{\text{sub.p}+1.\text{sup.k}}$, the exemplary methods only map $H_{\text{sub.p}+1.\text{sup.k}}$ to $G_{\text{sub.p}+1.\text{sup.k}-1}$ as its attributes, and then employ another VGAE **714** layer to convert the attributed graph to the state-dependent embedding.

(101) This process can be defined as:

$$(102) \quad \hat{Z}_{p+1}^k = g(A_{p+1}^{k-1}, H_{p+1}^k) \quad (8)$$

(103) Regarding the state-invariant decoder **720**, the goal of the state-invariant decoder is to learn the invariant causal relations across two system states. To recover the state-invariant part, the exemplary methods first feed $\{\tilde{Z}\}_{\text{sub.p}+1.\text{sup.k}}$ into the graph generator layer **722** to generate the corresponding state-invariant graph $\hat{G}_{\text{sub.p}+1.\text{sup.k}}$, which is defined as:

$$(104) \quad \hat{G}_{p+1}^k = \text{Sigmoid}(\tilde{Z}_{p+1}^k \cdot \text{Math. } \tilde{Z}_{p+1}^k) \quad (9)$$

(105) where Sigmoid is an activation function and $(\cdot)'$ is a transpose operation. But since this process constructs the graph using the state-invariant embeddings only, it can't guarantee that the state-invariant causal relationships are shown accurately in this graph. To overcome this issue, two optimization objectives must be met, that is, making $\hat{G}_{\text{sub.p}+1.\text{sup.k}}$ as similar to the previous causal graph $G_{\text{sub.p}+1.\text{sup.k}-1}$ as possible and fitting $\hat{G}_{\text{sub.p}+1.\text{sup.k}}$ to both the previous and new state data batches. To achieve the first objective, the reconstruction loss $L_{\text{sub.G}}$ is minimized, which is defined as:

$$(106) \quad L_{\hat{G}} = \text{Math. } \hat{A}_{p+1}^k - A_{p+1}^{k-1} \cdot \text{Math.} \quad (10)$$

(107) where $\hat{A}_{p+1.\text{sup}.k}$ and $A_{p+1.\text{sup}.k-1}$ are the adjacency matrices of $\hat{G}_{p+1.\text{sup}.k}$ and $G_{p+1.\text{sup}.k-1}$, respectively. To achieve the second objective, the exemplary methods fit the graph and data with a structural vector autoregressive (SVAR) model. More specifically, given the time-lagged data of the previous state $X_{p.\text{sup}.k}$ and the current new batch $X_{p+1.\text{sup}.k}$, the SVAR-based predictive equations can be defined as:

$$(108) \quad \begin{cases} \tilde{X}_p = \tilde{X}_p \cdot \text{Math} \cdot \hat{A}_{p+1}^k + \bar{X}_p \cdot \text{Math} \cdot \hat{D}_{p+1}^k + \tilde{o}_p \\ X_{p+1}^k = X_{p+1}^k \cdot \text{Math} \cdot \hat{A}_{p+1}^k + \bar{X}_{p+1} \cdot \text{Math} \cdot \hat{D}_{p+1}^k + \tilde{o}_{p+1}^k \end{cases} \quad (11)$$

(109) where $\tilde{o}_{p.\text{sup}.k}$ and $\tilde{o}_{p+1.\text{sup}.k}$, are vectors of centered error variables; $\hat{A}_{p+1.\text{sup}.k}$ is used to capture causal relations among system entities, and the weight matrix $\{\text{circumflex over (D)}\}_{p+1.\text{sup}.k}$ is used to model the contribution of time-lagged data for the predictive task. $\hat{A}_{p+1.\text{sup}.k}$ and $\{\text{circumflex over (D)}\}_{p+1.\text{sup}.k}$ are used to predict both the past state data and the current batch of data.

(110) To ensure the accuracy of learned causal structures, the exemplary methods minimize two predictive errors $L_{\tilde{p}}$ and $L_{\text{circumflex over (p)}}$, which are defined as:

$$(111) \quad \begin{cases} L_{\tilde{p}} = \text{Math} \cdot \tilde{X}_p - (\tilde{X}_p \cdot \text{Math} \cdot \hat{A}_{p+1}^k + \bar{X}_p \cdot \text{Math} \cdot \hat{D}_{p+1}^k) \cdot \text{Math} \cdot \\ L_{\tilde{p}} = \text{Math} \cdot X_{p+1}^k - (X_{p+1}^k \cdot \text{Math} \cdot \hat{A}_{p+1}^k + \bar{X}_{p+1} \cdot \text{Math} \cdot \hat{D}_{p+1}^k) \cdot \text{Math} \cdot \end{cases} \quad (12)$$

(112) Regarding the state-dependent decoder **740**, the goal of the state-dependent decoder is to learn the new causal relations introduced by the new batch of data. Similar to the learning process of the state-invariant decoder, the exemplary methods first generate the state-dependent graph $\check{G}_{p+1.\text{sup}.k}$ by applying the same strategy on the embedding $\check{Z}_{p+1.\text{sup}.k}$, which is defined as:

$$(113) \quad G_{p+1}^k = \text{Sigmoid}(Z_{p+1}^k \cdot \text{Math} \cdot Z_{p+1}^{k'}) \quad (13)$$

(114) To ensure the causal graph $\check{G}_{p+1.\text{sup}.k}$ is brought by the new batch of data $\check{X}_{p+1.\text{sup}.k}$, two optimization objectives must be met, that is, making $\check{G}_{p+1.\text{sup}.k}$ as similar to the complement of the previous causal graph as possible and fitting $\check{G}_{p+1.\text{sup}.k}$ to the new batch of data. To achieve the first objective, the reconstruction loss L_a is minimized, which is defined as:

$$(115) \quad L_G = \text{Math} \cdot A_{p+1}^k - (:A_{p+1}^{k-1}) \cdot \text{Math} \cdot \quad (14)$$

(116) where $(:A_{p+1.\text{sup}.k-1})$ refers to the inversion of each element in the adjacency matrix $A_{p+1.\text{sup}.k-1}$ and $\hat{A}_{p+1.\text{sup}.k}$, is the adjacency matrix of $\check{G}_{p+1.\text{sup}.k}$.

(117) To achieve the second objective, a predictive equation using SVAR is defined:

$$(118) \quad Y_{p+1}^k = X_{p+1}^k \cdot \text{Math} \cdot \hat{A}_{p+1}^k + \bar{X}_{p+1} \cdot \text{Math} \cdot \hat{D}_{p+1}^k + \tilde{o} \quad (15)$$

(119) where $\hat{A}_{p+1.\text{sup}.k}$ captures the new causal relations introduced by the new data batch $\check{X}_{p+1.\text{sup}.k}$ and $\hat{D}_{p+1.\text{sup}.k}$ is to model the contribution of time-lagged data for prediction. To ensure the accuracy of learned causal structures, the predictive error $L_{\text{sub.p}}$ is minimized, which is defined as:

$$(120) \quad L_p = \text{Math} \cdot X_{p+1}^k - (X_{p+1}^k \cdot \text{Math} \cdot \hat{A}_{p+1}^k + \bar{X}_{p+1} \cdot \text{Math} \cdot \hat{D}_{p+1}^k) \cdot \text{Math} \cdot \quad (16)$$

(121) Regarding the causal graph fusion **760**, from the state-invariant decoder and state-dependent decoder, the exemplary methods can obtain the state-invariant causal graph $\hat{G}_{p+1.\text{sup}.k}$ and the state-dependent causal graph $\check{G}_{p+1.\text{sup}.k}$, respectively. To generate the causal graph $G_{p+1.\text{sup}.k}$ for the current batch of data, simple addition will not work because it may result in dense and cyclical graphs. Here, a new graph fusion layer **762** is presented to fuse the two causal graphs, which can be formulated as follows:

$$(122) \quad A_{p+1}^k = \text{RELU}(\tanh(\hat{A}_{p+1}^k \cdot \text{Math} \cdot A_{p+1}^{k'} - A_{p+1}^k \cdot \text{Math} \cdot A_{p+1}^{k'})) \quad (17)$$

(123) where $A_{p+1.\text{sup}.k}$ is the adjacency matrix of $G_{p+1.\text{sup}.k}$. The subtraction term, $\tanh$, and RELU activation functions may regularize the adjacency matrix so that if the element in $A_{p+1.\text{sup}.k}$ is positive, its diagonal counterpart element will be zero. To strictly force the $G_{p+1.\text{sup}.k}$ to be unidirectional and acyclic, the exemplary methods adopt the following exponential trace function as constraints.

$$(124) \quad h(A_{p+1}^k) = \text{tr}(e^{A_{p+1}^k \circ A_{p+1}^k}) - M \quad (18)$$

(125) where $\circ$ is the Hadamard product of two matrices and M is the number of nodes (e.g., system entities). This function satisfies $h(A_{p+1.\text{sup}.k})=0$ if and only if $A_{p+1.\text{sup}.k}$ is acyclic.

(126) Regarding optimization, to generate a robust casual graph for the new data batch, the exemplary methods jointly optimize all the preceding loss functions.

(127) Thus, the final optimization objective is defined as:

(128)

$$L = L_{\hat{G}} + L_G + L_{\tilde{p}} + L_{\hat{p}} + L_p + \lambda_1 \cdot \text{Math.}(\|\hat{A}_{p+1}^k - A_{p+1}^k\|_1) + \lambda_2 \cdot \text{Math.}h(A_{p+1}^k) \quad (19)$$

(129) where $\|\cdot\|_1$ is the L1-norm, which is used to increase the sparsity of $\hat{G}_{p+1,k}$ and $\tilde{G}_{p+1,k}$ to reduce the computational cost, where λ_1 and λ_2 control the penalized degree of regularization items.

(130) Regarding model convergence, the discovered causal structure and the associated root cause list may gradually converge as the number of new data batches increases. So the exemplary methods incorporate them as an indicator to automatically terminate the online RCA to avoid unnecessary computing resource waste.

(131) It is assumed that two consecutive causal graphs are $G_{p+1,K-1}$, $G_{p+1,K}$, and the associated root cause lists are $I_{p+1,K-1}$, $I_{p+1,K}$.

(132) The node set of the two causal graphs is fixed. The edge distribution should be comparable when the causal graph converges. Thus, the exemplary methods define the graph similarity $\text{custom character.sub.G}$ using the Jensen-Shannon divergence as follows:

$$\text{custom character.sub.G} = 1 - \text{JS}(P(G_{p+1}^{K-1}), P(G_{p+1}^K)) \quad (20)$$

(134) where $P(\cdot)$ refers to the edge distribution of the corresponding graph. $\text{custom character.sub.G}$ has a value range of [0:1]. The greater the value of $\text{custom character.sub.G}$ is, the closer the two graphs are.

(135) The exemplary methods use the rank-biased overlap metric (RBO) to calculate the similarity between two root cause lists in order to fully account for the changing trend of root cause rank. The ranked list similarity $\text{custom character.sub.1}$ is defined as:

$$\text{custom character.sub.1} = \text{RBO}(I_{p+1}^{K-1}, I_{p+1}^K) \quad (21)$$

(137) $\text{custom character.sub.1}$ has a value range of [0:1]. The greater the value of $\text{custom character.sub.1}$ is, the more similar the two root cause lists are. The exemplary methods use a hyperparameter $\alpha \in [0:1]$ to integrate $\text{custom character.sub.G}$ and $\text{custom character.sub.1}$, defined as:

$$\text{custom character} = \alpha \cdot \text{custom character.sub.G} + (1 - \alpha) \cdot \text{custom character.sub.1} \quad (22)$$

(139) The online RCA process may stop when custom character is less than a threshold.

(140) Regarding network propagation-based root cause localization, after obtaining the causal graph $G_{p+1,k}$, there are two kinds of nodes, that is, system entities and system KPI in the graph. However, the system entities linked to the KPI may not always be the root causes. This is because the malfunctioning effects spread to neighboring entities starting from the root causes. The exemplary methods present a random walk-based method for capturing such patterns and more precisely locating root causes. For simplicity, the exemplary methods directly use G to represent $G_{p+1,k}$. To trace back the root causes, the exemplary methods first transpose the learned causal graph to get G^T , then adopt a random walk with restarts on the transposed causal graph to estimate the probability score of each entity by starting from the KPI node.

(141) Specifically, it is assumed that the transition probabilities of a particle on the transposed structure may be represented by H , which has the same shape as the adjacency matrix of G^T . Each element in H indicates the transition probability between any two nodes. Imagine that from the KPI node, a particle begins to visit the causal structure. It jumps to any one node with a probability value $\phi \in [0,1]$ or stays at the original position with $1-\phi$. The higher the value of ϕ is, the more possible the jumping behavior happens. Specifically, if the particle moves from node i to node j , the moving probability in H should be updated by:

$$H[i,j] = (1 - \phi)A^*[i,j] / \sum_{i=1}^M A^*[i, \cdot] \quad (23)$$

(143) where A^T is the adjacency matrix of G^T . During the visiting exploration process, the exemplary methods may restart from the KPI node to revisit other entities with the probability $\phi \in [0,1]$. Thus, the visiting probability transition equation of the random walk with restarts can be formulated as:

$$q_{t+1} = (1 - \phi) \cdot q_t + \phi \cdot q_{\text{KPI}} \quad (24)$$

(145) where q_t and q_{t+1} are the visiting probability distributions at the t and $t+1$ steps, respectively. q_{KPI} is the initial visiting probability distribution at the initial step. When the visiting probability distribution converges, the probability scores of the nodes are used as their causal scores to rank them. The top- K ranked nodes are the most likely root causes of the associated system fault.

(146) In conclusion, the exemplary embodiments introduce an online root cause analysis framework, which includes online trigger point detection and incremental causal discovery. For the first part, the exemplary methods propose a non-linear kernel-based subspace learning method for detecting the system state change points. A monitoring agent is designed based on JMeter and Openshift/Prometheus to collect the performance KPI and metrics data from the whole system and the underlying system components. To learn the representation of the system state, the exemplary methods use a time-lagged trajectory matrix to collect data and leverage non-linear kernel-based matrix decomposition to learn a robust subspace. To detect the system change point accurately, the exemplary methods employ CUSUM to

accumulate the distribution error until achieving the threshold. For the second part, the exemplary methods propose a disentangle graph learning-based incremental causal discovery framework. To efficiently learn the causal graph, the exemplary methods present a two-branch disentangle learning framework. The first branch is to learn the state-invariant causal information and the second branch is to learn the state-dependent causal information. To merge the two branch results, the exemplary methods introduce a new graph learning layer that can produce sparse and directed acyclic causal graphs. To justify the quality of the learned causal graph, the exemplary methods utilize a prediction layer to predict the future time-series data based on the learned causal graph.

(147) The proposed online trigger point detection method extends the traditional linear-based change point detection method to non-linear. The matrix-decomposition-based subspace learning ensures the computational efficiency of online trigger point detection. Hence, the detection algorithm can be conducted in a short delay time, which is the cornerstone of the online root cause analysis. The proposed causal discovery first utilizes the disentangle-graph learning method to efficiently learn causal graphs over system state changes. The disentangle learning process reduces the computational cost of traditional causal discovery.

(148) The proposed method is more accurate (higher quality) on root cause identification, hence the generated root causes will have less false positives and false negatives based on the discovered results of hierarchical root causes. Traditional root cause analysis methods are conducted offline. When finding a system failure, operators must recollect historical data and rebuild the causal discovery model. In comparison to such approaches, the exemplary method is an online root cause analysis that can automatically detect the early phenomenon of system failures and incrementally update learned causal graphs to effectively locate root causes. The proposed method can detect root causes of system failures in real-time.

(149) FIG. 8 is a block/flow diagram of an exemplary online incident backtrack and system recovery engine, in accordance with embodiments of the present invention.

(150) The computer systems **802** are monitored by agents. The data is provided two big data processing middleware **804**. A user **806** performs attack forensic analysis.

(151) FIG. 9 is a block/flow diagram of an exemplary incremental causal graph learning for attack forensics in an enterprise system, in accordance with embodiments of the present invention.

(152) Regarding the enterprise system monitoring and data collection, the agent **818** collects the enterprise system data by employing a logging agent or tool such as Event Tracing for Windows (ETW) and strace. Two types of monitored data are used in the attack forensic analysis engine **810**. The exemplary methods collect metrics such as response time, CPU usage, memory usage, and network traffic and collect logs related to different processes such as process creation, file operation, network operations, etc.

(153) Regarding data preprocessing, for metrics data **820**, it is possible that there are different levels of data such as high-level (e.g., computer/server level) system metric data and low-level (e.g., process-level) system metric data and for each level, there are different metrics (like CPU usage, memory usage, etc.). The exemplary methods extract the data of the same level and same metric to construct the multivariate time-series with columns representing system entities (e.g., processes) and rows representing different timestamps.

(154) Regarding the online trigger point detection **300**, the goal is to detect the system transition points to trigger incremental causal graph learning **812**. The exemplary methods model the underlying dynamics of system entity metrics (e.g., multivariate time series observations) through the Multivariate Singular Spectrum Analysis (MSSA) model.

(155) Trigger points are provided to the incremental disentangled causal graph learning **400**. Thereafter, at block **830**, backtracking from attack detection point using a dependency graph to locate an attack origin is performed. At block **832**, the attack data from the backtracking is analyzed to stop attacks and/or prevent future attacks. The attack data can be displayed on a visualization module **834**, which is a graphical user interface (GUI). At block **814**, the benefits of the attack forensic analysis using the online incident backtrack and system recovery engine are presented. Such benefits include at least online early attack detection and efficient online causal dependency graph generation.

(156) FIG. 10 is a block/flow diagram of an exemplary architecture of automatic security intelligence **850**, in accordance with embodiments of the present invention.

(157) Traditionally, enterprises protect themselves by trying to keep the attackers outside through perimeter defenses such as firewalls and intrusion prevention systems (IPS). However, given the sophistication of modern attacks, such as drive-by download, phishing emails, contaminated mobile devices, and insider attacks, successful intrusions and compromises are almost unavoidable. For example, recently, there have been many high-profile data breaches. Therefore, in the real world, the fundamental assumption that enterprise security management is simply to prevent attackers from entering into an enterprise no longer holds.

(158) Given such situation, attack forensics become very important. In attack forensics, operators backward trace back the origin of the attack given a detection point based on the multi-modality data including metrics and log data. For example, if a malware instance is located inside a computer, the operators want to trace back how such a malware got into the system. This helps system administrators identify and patch the root causes of the intrusion and strengthen the enterprise's security.

(159) Previous data-driven studies have focused primarily on developing effective offline methods. A key component of many data-driven offline attack forensics algorithms, especially the backtracking-based attack forensics methods is to generate causal dependency graph that profiles the causal relations between system entities based on historical data, so that the operators can trace back the attack origin based on the built causal graph.

(160) However, there are three limitations in the traditional offline backtracking-based attack forensics workflow. First, for a new intrusion or attack, operators need to retrain/rebuild the model from scratch. Second, the causal graph generation component is often time-consuming and requires a large amount of historical data to train a robust model. Third, it often requires the operators to manually initiate the process when they observe an attack. As a result, it's often too late to mitigate any damage or loss caused by an intrusion or attack.

(161) Enlightened by incremental learning, the exemplary embodiments of the present invention aim to incrementally update the causal dependency graph from streaming system monitoring data for accurately identifying an attack origin when an intrusion or attack occurs.

(162) Referring back to FIG. 10, the agent **860** is installed in each machine of the enterprise network **880** to collect operational data. The backend servers **870** receive the data from the agents **860**, pre-process the data, and send them to the analytic server **875**. The analytic server **875** runs the security application programs to analyze the data. The incident backtrack and system recovery engine **896** is a major application to track and analyze security incidents, identify the source of the attack, and determine the extent of the damage. It is also used to recover lost or corrupted data and restore systems to their pre-attack state. The incident backtrack and system recovery engine **896** can be used by security analysts, incident response teams, and system administrators to respond to security incidents and minimize the impact on the affected systems and networks quickly and effectively. The security applications **890** can further include intrusion detection **892**, security policy compliance assessment **894**, and centralized threat and security query **898**.

(163) Therefore, the matrix-decomposition-based subspace learning ensures the computational efficiency of online trigger point detection. Hence, the detection algorithm can be conducted in a short delay time, which is the cornerstone of the online attack forensics. The proposed causal discovery first utilizes the disentangle-graph learning concept to efficiently learn causal graphs over system state changes. The disentangle learning process reduces the computational cost of traditional causal dependency graph generation. The proposed method is more accurate (higher quality) on attack origin identification, and, hence, the generated attack origins will have less false positives and false negatives. Traditional attack forensics methods are conducted offline. When finding an intrusion or attack, operators must recollect historical data and rebuild the causal dependency graphs. In comparison to such approaches, the exemplary method is related to online attack forensics for automatically detecting the early phenomenon of an attack and incrementally updating the learned causal graphs for effectively locating attack origins. The proposed method can detect an attack origin of intrusions or attacks in real time.

(164) FIG. 11 is an exemplary processing system for online incident backtrack and system recovery, in accordance with embodiments of the present invention.

(165) The processing system includes at least one processor (CPU) **904** operatively coupled to other components via a system bus **902**. A GPU **905**, a cache **906**, a Read Only Memory (ROM) **908**, a Random Access Memory (RAM) **910**, an input/output (I/O) adapter **920**, a network adapter **930**, a user interface adapter **940**, and a display adapter **950**, are operatively coupled to the system bus **902**. Additionally, CORAL **900** is presented including online trigger point detection **300** and incremental causal discovery **400** for attack forensics analysis **830**.

(166) A storage device **922** is operatively coupled to system bus **902** by the I/O adapter **920**. The storage device **922** can be any of a disk storage device (e.g., a magnetic or optical disk storage device), a solid-state magnetic device, and so forth.

(167) A transceiver **932** is operatively coupled to system bus **902** by network adapter **930**.

(168) User input devices **942** are operatively coupled to system bus **902** by user interface adapter **940**. The user input devices **942** can be any of a keyboard, a mouse, a keypad, an image capture device, a motion sensing device, a microphone, a device incorporating the functionality of at least two of the preceding devices, and so forth. Of course, other types of input devices can also be used, while maintaining the spirit of the present invention. The user input devices **942** can be the same type of user input device or different types of user input devices. The user input devices **942** are used to input and output information to and from the processing system.

(169) A display device **952** is operatively coupled to system bus **902** by display adapter **950**.

(170) Of course, the processing system may also include other elements (not shown), as readily contemplated by one of skill in the art, as well as omit certain elements. For example, various other input devices and/or output devices can be included in the system, depending upon the particular implementation of the same, as readily understood by one of ordinary skill in the art. For example, various types of wireless and/or wired input and/or output devices can be used. Moreover, additional processors, controllers, memories, and so forth, in various configurations can also be utilized as readily appreciated by one of ordinary skill in the art. These and other variations of the processing system are readily contemplated by one of ordinary skill in the art given the teachings of the present invention provided herein.

(171) FIG. 12 is a block/flow diagram of an exemplary method for online incident backtrack and system recovery, in

accordance with embodiments of the present invention.

(172) At block **1001**, detect a trigger point from entity metrics data and key performance indicator (KPI) data.

(173) At block **1003**, generate a learned causal graph by fusing a state-invariant causal graph with a state-dependent causal graph.

(174) At block **1005**, backtrack from an attack detection point, via an incident backtrack and system recovery component, by using the learned causal graph to identify an attack origin when an intrusion or attack occurs.

(175) At block **1007**, display data relating to the attack origin on a visualization display for user analysis.

(176) As used herein, the terms “data,” “content,” “information” and similar terms can be used interchangeably to refer to data capable of being captured, transmitted, received, displayed and/or stored in accordance with various example embodiments. Thus, use of any such terms should not be taken to limit the spirit and scope of the disclosure. Further, where a computing device is described herein to receive data from another computing device, the data can be received directly from the another computing device or can be received indirectly via one or more intermediary computing devices, such as, for example, one or more servers, relays, routers, network access points, base stations, and/or the like. Similarly, where a computing device is described herein to send data to another computing device, the data can be sent directly to the another computing device or can be sent indirectly via one or more intermediary computing devices, such as, for example, one or more servers, relays, routers, network access points, base stations, and/or the like.

(177) As will be appreciated by one skilled in the art, aspects of the present invention may be embodied as a system, method or computer program product. Accordingly, aspects of the present invention may take the form of an entirely hardware embodiment, an entirely software embodiment (including firmware, resident software, micro-code, etc.) or an embodiment combining software and hardware aspects that may all generally be referred to herein as a “circuit,” “module,” “calculator,” “device,” or “system.” Furthermore, aspects of the present invention may take the form of a computer program product embodied in one or more computer readable medium(s) having computer readable program code embodied thereon.

(178) Any combination of one or more computer readable medium(s) may be utilized. The computer readable medium may be a computer readable signal medium or a computer readable storage medium. A computer readable storage medium may be, for example, but not limited to, an electronic, magnetic, optical, electromagnetic, infrared, or semiconductor system, apparatus, or device, or any suitable combination of the foregoing. More specific examples (a non-exhaustive list) of the computer readable storage medium would include the following: an electrical connection having one or more wires, a portable computer diskette, a hard disk, a random access memory (RAM), a read-only memory (ROM), an erasable programmable read-only memory (EPROM or Flash memory), an optical fiber, a portable compact disc read-only memory (CD-ROM), an optical data storage device, a magnetic data storage device, or any suitable combination of the foregoing. In the context of this document, a computer readable storage medium may be any tangible medium that can include, or store a program for use by or in connection with an instruction execution system, apparatus, or device.

(179) A computer readable signal medium may include a propagated data signal with computer readable program code embodied therein, for example, in baseband or as part of a carrier wave. Such a propagated signal may take any of a variety of forms, including, but not limited to, electro-magnetic, optical, or any suitable combination thereof. A computer readable signal medium may be any computer readable medium that is not a computer readable storage medium and that can communicate, propagate, or transport a program for use by or in connection with an instruction execution system, apparatus, or device.

(180) Program code embodied on a computer readable medium may be transmitted using any appropriate medium, including but not limited to wireless, wireline, optical fiber cable, RF, etc., or any suitable combination of the foregoing.

(181) Computer program code for carrying out operations for aspects of the present invention may be written in any combination of one or more programming languages, including an object oriented programming language such as Java, Smalltalk, C++ or the like and conventional procedural programming languages, such as the “C” programming language or similar programming languages. The program code may execute entirely on the user's computer, partly on the user's computer, as a stand-alone software package, partly on the user's computer and partly on a remote computer or entirely on the remote computer or server. In the latter scenario, the remote computer may be connected to the user's computer through any type of network, including a local area network (LAN) or a wide area network (WAN), or the connection may be made to an external computer (for example, through the Internet using an Internet Service Provider).

(182) Aspects of the present invention are described below with reference to flowchart illustrations and/or block diagrams of methods, apparatus (systems) and computer program products according to embodiments of the present invention. It will be understood that each block of the flowchart illustrations and/or block diagrams, and combinations of blocks in the flowchart illustrations and/or block diagrams, can be implemented by computer program instructions. These computer program instructions may be provided to a processor of a general-purpose computer, special purpose computer, or other programmable data processing apparatus to produce a machine, such that the instructions, which

executed via the processor of the computer or other programmable data processing apparatus, create means for implementing the functions/acts specified in the flowchart and/or block diagram block or blocks or modules.

(183) These computer program instructions may also be stored in a computer readable medium that can direct a computer, other programmable data processing apparatus, or other devices to function in a particular manner, such that the instructions stored in the computer readable medium produce an article of manufacture including instructions which implement the function/act specified in the flowchart and/or block diagram block or blocks or modules.

(184) The computer program instructions may also be loaded onto a computer, other programmable data processing apparatus, or other devices to cause a series of operational steps to be performed on the computer, other programmable apparatus or other devices to produce a computer implemented process such that the instructions which execute on the computer or other programmable apparatus provide processes for implementing the functions/acts specified in the flowchart and/or block diagram block or blocks or modules.

(185) It is to be appreciated that the term “processor” as used herein is intended to include any processing device, such as, for example, one that includes a CPU (central processing unit) and/or other processing circuitry. It is also to be understood that the term “processor” may refer to more than one processing device and that various elements associated with a processing device may be shared by other processing devices.

(186) The term “memory” as used herein is intended to include memory associated with a processor or CPU, such as, for example, RAM, ROM, a fixed memory device (e.g., hard drive), a removable memory device (e.g., diskette), flash memory, etc. Such memory may be considered a computer readable storage medium.

(187) In addition, the phrase “input/output devices” or “I/O devices” as used herein is intended to include, for example, one or more input devices (e.g., keyboard, mouse, scanner, etc.) for entering data to the processing unit, and/or one or more output devices (e.g., speaker, display, printer, etc.) for presenting results associated with the processing unit.

(188) The foregoing is to be understood as being in every respect illustrative and exemplary, but not restrictive, and the scope of the invention disclosed herein is not to be determined from the Detailed Description, but rather from the claims as interpreted according to the full breadth permitted by the patent laws. It is to be understood that the embodiments shown and described herein are only illustrative of the principles of the present invention and that those skilled in the art may implement various modifications without departing from the scope and spirit of the invention. Those skilled in the art could implement various other feature combinations without departing from the scope and spirit of the invention. Having thus described aspects of the invention, with the details and particularity required by the patent laws, what is claimed and desired protected by Letters Patent is set forth in the appended claims.

Claims

1. A computer-implemented method for identifying attack origins, the method comprising: detecting a trigger point from entity metrics data and key performance indicator (KPI) data; generating a learned causal graph by fusing a state-invariant causal graph with a state-dependent causal graph; backtracking from an attack detection point, via an incident backtrack and system recovery component, by using the learned causal graph to identify an attack origin in response to an intrusion or an attack occurring; and displaying data relating to the attack origin on a visualization display for user analysis.
2. The computer-implemented method of claim 1, wherein the trigger point detection involves constructing an initial system state space by using a time-lagged trajectory matrix.
3. The computer-implemented method of claim 2, wherein the time-lagged trajectory matrix leverages non-linear kernel-based matrix decomposition to learn a robust subspace.
4. The computer-implemented method of claim 1, wherein a disentangle graph learning-based incremental causal discovery framework is employed to generate the learned causal graph.
5. The computer-implemented method of claim 1, wherein a prediction layer is used to predict future time-series data on the learned causal graph.
6. The computer-implemented method of claim 1, wherein the learned causal graph is incrementally updated.
7. The computer-implemented method of claim 1, wherein a state-invariant decoder is employed to learn invariant causal relations across two system states and wherein a state-dependent decoder is employed to learn new causal relations introduced by a new batch data.
8. A computer program product for identifying attack origins, the computer program product comprising a non-transitory computer readable storage medium having program instructions embodied therewith, the program instructions executable by a computer to cause the computer to perform a method comprising: detecting a trigger point from entity metrics data and key performance indicator (KPI) data; generating a learned causal graph by fusing a state-invariant causal graph with a state-dependent causal graph; backtracking from an attack detection point, via an incident backtrack and system recovery component, by using the learned causal graph to identify an attack origin in response to an intrusion or an attack occurring; and displaying data relating to the attack origin on a visualization display for user analysis.

9. The computer program product of claim 8, wherein the trigger point detection involves constructing an initial system state space by using a time-lagged trajectory matrix.
10. The computer program product of claim 9, wherein the time-lagged trajectory matrix leverages non-linear kernel-based matrix decomposition to learn a robust subspace.
11. The computer program product of claim 8, wherein a disentangle graph learning-based incremental causal discovery framework is employed to generate the learned causal graph.
12. The computer program product of claim 8, wherein a prediction layer is used to predict future time-series data on the learned causal graph.
13. The computer program product of claim 8, wherein the learned causal graph is incrementally updated.
14. The computer program product of claim 8, wherein a state-invariant decoder is employed to learn invariant causal relations across two system states and wherein a state-dependent decoder is employed to learn new causal relations introduced by a new batch data.
15. A computer processing system for identifying attack origins, comprising: a memory device for storing program code; and a processor device, operatively coupled to the memory device, for running the program code to: detect a trigger point from entity metrics data and key performance indicator (KPI) data; generate a learned causal graph by fusing a state-invariant causal graph with a state-dependent causal graph; backtrack from an attack detection point, via an incident backtrack and system recovery component, by using the learned causal graph to identify an attack origin in response to an intrusion or an attack occurring; and display data relating to the attack origin on a visualization display for user analysis.
16. The computer processing system of claim 15, wherein the trigger point detection involves constructing an initial system state space by using a time-lagged trajectory matrix.
17. The computer processing system of claim 16, wherein the time-lagged trajectory matrix leverages non-linear kernel-based matrix decomposition to learn a robust subspace.
18. The computer processing system of claim 15, wherein a disentangle graph learning-based incremental causal discovery framework is employed to generate the learned causal graph.
19. The computer processing system of claim 15, wherein a prediction layer is used to predict future time-series data on the learned causal graph.
20. The computer processing system of claim 15, wherein the learned causal graph is incrementally updated.
-